

Day 34: DNS: From Infrastructure Service to Enumeration Target

Security+ Lesson 3

Hashing vs Encryption vs Encoding vs Digital Signatures

Encryption

Encryption protects data so unauthorized parties cannot understand it.

Basic idea: Readable plaintext -> Encryption + Key -> Ciphertext

Authorized parties use the appropriate key/process to recover the plaintext. Encryption is therefore reversible when you possess the required cryptographic key. Think primarily: protect confidentiality.

Hashing

A hash function takes input and produces a fixed-size output.

Input -> Hash function -> Hash value

We generally do not decrypt a hash back into the original input. So Encryption DNE Hashing.

Useful purposes: integrity checking, password-storage systems, file verification

Hashing a password does not make weak passwords magically safe. Attackers can still guess passwords, hash the guesses, and compare results.

Salting

A salt is additional random data incorporated into password hashing. Imagine two people both use: Bear123!

Without salting, identical inputs lead to identical stored password-hash outputs under the same hashing scheme.

With unique salts: Bear123! + Salt A -> Hash A

Bear123! + Salt B -> Hash B

Encoding

Not security by itself. It changes data representation so another system/process can interpret or transport it. For example, something may be converted to Base64. Someone who understands the encoding can simply decode it. So Encoded is not the same as Encrypted.

Digital Signatures

Think of the concept of non-repudiation. A digital signature is designed to help establish: authenticity, integrity, and evidence associated with a signing identity.

Data -> Cryptographic signing operation -> Digital signature

Recipient Verifies -> Was the signed data altered? Was it signed using the expected key?

Phase 1 Recall: DNS From Memory

What problem does DNS solve?

It fixes the problem of finding computers and services by name instead of having to know their IP addresses.

What does an A record do?

An A record maps a hostname to an IPv4 address.

SRV-DC01.kevinlab.local -> 192.168.1.10

What does a SRV record do?

SRV records tell a computer where a specific service is located on the network. SRV record = Service -> Server + Port

Why does CLIENT01 have to use SRV-DC01 as DNS?

CLIENT01 needs DNS that knows about the Active Directory domain, and SRV-DC01 hosts that DNS information.

What does **_ldap._tcp.dc._msdcs.kevinlab.local** mean?

This is the DNS SRV record name Active Directory clients use to find a DC that provides LDAP for the domain.

Why did being able to ping the DC not prove CLIENT01's AD configuration was correct?

This only tests network reachability; it has nothing to do with AD setup/configuration.

DNS as an Assessor

DNS can help an assessor map infrastructure. It may reveal: hostnames, IP addresses, Domain Controllers, mail systems, service locations, Kerberos services, LDAP services, Global Catalog systems

When I discovered: 53/tcp open on SRV-DC01, the question becomes what does this DNS server know about the environment?

Meet dig

Kali commonly has: dig, a DNS-query tool.

Structure: **dig @DNS_SERVER NAME RECORD_TYPE**

Ex: **dig @192.168.1.236 kevinlab.local A**

Breakdown:

- Dig: dns query tool
- @192.168.1.236: ask THIS DNS server specifically
- Kevinlab.local: name being queried
- A: record type requested

Understanding dig output:

Question Section: What did we ask?

Answer Section: What answer did the DNS server return?

Server: Which DNS server gave us the response?

Flags/status: Did the query succeed? Was it authoritative? Was recursion involved?

Guided Practical: Enumerating AD DNS

Server: SRV-DC01

Part 1: Basic domain query

Goal: Can the AD DNS server provide information about its own domain?

```
(nigist@kali)-[~]
$ nmap -Pn -p 53 -sV 192.168.1.236
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-20 09:59 EDT
Nmap scan report for 192.168.1.236
Host is up (0.0013s latency).

PORT      STATE SERVICE VERSION
53/tcp    open  domain Simple DNS Plus
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org
Nmap done: 1 IP address (1 host up) scanned in 6.46 seconds
```

```
(nigist@kali)-[~]
$ dig @192.168.1.236 kevinlab.local A

; <<>> DiG 9.19.21-1+b1-Debian <<>> @192.168.1.236 kevinlab.local A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 5677
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;kevinlab.local.                IN      A

;; ANSWER SECTION:
kevinlab.local.                600     IN      A      192.168.1.236

;; Query time: 0 msec
;; SERVER: 192.168.1.236#53(192.168.1.236) (UDP)
;; WHEN: Thu Aug 20 10:02:01 EDT 2026
;; MSG SIZE rcvd: 59
```

;; ->> HEADER <<- opcode: QUERY, status: NOERROR, id: 5677

Where status: NOERROR means the DNS query worked

Flags:

- qr: response to a query
- aa: authoritative answer
- rd: recursion was requested
- ra: the server supports recursion

ANSWER SECTION:

- Read as kevinlab.local resolves to 192.168.1.236
- Kevinlab.local is name, 600 is TTL

Part 2: Domain Controller Discovery

Now query the SRV record I have seen since Phase 1.

Try: **dig @192.168.1.236 _ldap._tcp.dc._msdcs.kevinlab.local SRV**

```
(nigist@kali)~  
$ dig @192.168.1.236 _ldap._tcp.dc._msdcs.kevinlab.local SRV  
  
; <<> DiG 9.19.21-1+b1-Debian <<> @192.168.1.236 _ldap._tcp.dc._msdcs.kevinlab.local  
SRV  
; (1 server found)  
;; global options: +cmd  
;; Got answer:  
;; WARNING: .local is reserved for Multicast DNS  
;; You are currently testing what happens when an mDNS query is leaked to DNS  
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 41123  
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 3  
  
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags:; udp: 4000  
;; QUESTION SECTION:  
_ldap._tcp.dc._msdcs.kevinlab.local. IN      SRV  
  
;; ANSWER SECTION:  
_ldap._tcp.dc._msdcs.kevinlab.local. 600 IN SRV 0 100 389 srv-dc01.kevinlab.local.  
  
;; ADDITIONAL SECTION:  
srv-dc01.kevinlab.local. 3600 IN      A      192.168.1.236  
srv-dc01.kevinlab.local. 3600 IN      AAAA    2600:4040:713e:c300:38b7:a55a:e2f7:4095  
  
;; Query time: 0 msec  
;; SERVER: 192.168.1.236#53(192.168.1.236) (UDP)  
;; WHEN: Thu Aug 20 10:12:24 EDT 2026  
;; MSG SIZE  rcvd: 151
```

The command basically means: Ask DNS Server 192.168.1.236 which server provides LDAP domain-controller service for kevinlab.local.

;; ANSWER SECTION:

_ldap._tcp.dc._msdcs.kevinlab.local. 600 IN SRV 0 100 389 srv-dc01.kevinlab.local.

ANSWER SECTION:

The domain controller providing LDAP over TCP is srv-dc01.kevinlab.local on port 389.

Part 3: Kerberos Service Discovery

Kerberos exists in AD. SRV records advertise services. DNS naming follows patterns.

The goal is to query for the domain's Kerberos SRV service.

Goal: Can DNS tell an assessor where Kerberos authentication services exist?

Command: **dig @192.168.1.236 _kerberos._tcp.kevinlab.local SRV**

```
(nigist@kali)-[~]
$ dig @192.168.1.236 _kerberos._tcp.kevinlab.local SRV

; <<>> DiG 9.19.21-1+b1-Debian <<>> @192.168.1.236 _kerberos._tcp.kevinlab.local SRV
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 52590
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 3

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;_kerberos._tcp.kevinlab.local. IN      SRV

;; ANSWER SECTION:
_kerberos._tcp.kevinlab.local. 600 IN      SRV      0 100 88 srv-dc01.kevinlab.local.

;; ADDITIONAL SECTION:
srv-dc01.kevinlab.local. 3600 IN      A        192.168.1.236
srv-dc01.kevinlab.local. 3600 IN      AAAA     2600:4040:713e:c300:38b7:a55a:e2f7:4095

;; Query time: 0 msec
;; SERVER: 192.168.1.236#53(192.168.1.236) (UDP)
;; WHEN: Thu Aug 20 10:18:51 EDT 2026
;; MSG SIZE rcvd: 145
```

Part 4: Resolve Discovered Hosts

SRV gave us the hostname: srv-dc01.kevinlab.local

Lets query its A record.

The chain: Domain -> Service Discovery -> Server Hostname -> Server IPv4 address

Command: **dig @192.168.1.236 srv-dc01.kevinlab.local A**

```
(nigist@kali)-[~]
$ dig @192.168.1.236 srv-dc01.kevinlab.local A

; <<>> DiG 9.19.21-1+b1-Debian <<>> @192.168.1.236 srv-dc01.kevinlab.local A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 38017
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4000
;; QUESTION SECTION:
;srv-dc01.kevinlab.local.      IN      A

;; ANSWER SECTION:
srv-dc01.kevinlab.local. 3600 IN      A        192.168.1.236

;; Query time: 0 msec
;; SERVER: 192.168.1.236#53(192.168.1.236) (UDP)
;; WHEN: Thu Aug 20 10:21:54 EDT 2026
;; MSG SIZE rcvd: 68
```

Part 5: Test a DNS Zone Transfer

DNS servers sometimes need to replicate zone information between authorized DNS servers. A DNS zone transfer can copy records from one DNS server to another. But imagine an arbitrary network client could request: Give me every DNS record for kevinlab.local.

So...an assessor may test whether a zone transfer is improperly allowed.

Dig calls a full zone transfer: **AXFR**.

Components:

- Tool: dig
- Server: SRV-DC01
- Zone: kevinlab.local
- Record/query type: AXFR

Command: **dig @192.168.1.236 kevinlab.local AXFR**



```
(nigist@kali)-[~]  
$ dig @192.168.1.236 kevinlab.local AXFR  
  
; <<>> DiG 9.19.21-1+b1-Debian <<>> @192.168.1.236 kevinlab.local AXFR  
; (1 server found)  
;; global options: +cmd  
; Transfer failed.
```

This should be good that it failed!